

```
!pip install langgraph
```

```
Requirement already satisfied: langgraph in /usr/local/lib/python3.12/dist-packages (0.6.8)
Requirement already satisfied: langchain-core>=0.1 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0.3.77)
Requirement already satisfied: langgraph-checkpoint<3.0.0,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (2.1.0)
Requirement already satisfied: langgraph-prebuilt<0.7.0,>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0.6.0)
Requirement already satisfied: langgraph-sdk<0.3.0,>=0.2.2 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0.2.9)
Requirement already satisfied: pydantic>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langgraph) (2.11.9)
Requirement already satisfied: xxhash>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (3.5.0)
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (0.1.12)
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (8.1.0)
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (1.33.0)
Requirement already satisfied: PyYAML<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (6.0.2)
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (4.12.0)
Requirement already satisfied: packaging<26.0.0,>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->langchain-core) (23.2.0)
Requirement already satisfied: ormsgpack>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from langgraph-checkpoint<3.0.0,>=2.1.0->langgraph-checkpoint) (1.10.0)
Requirement already satisfied: httpx>=0.25.2 in /usr/local/lib/python3.12/dist-packages (from langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (0.25.2)
Requirement already satisfied: orjson>=3.10.1 in /usr/local/lib/python3.12/dist-packages (from langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (3.10.1)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->pydantic) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->pydantic) (0.4.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (4.6.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (2025.1.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (3.10)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (0.14.0)
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0,>=1.33.0->langgraph-checkpoint<3.0.0,>=2.1.0->langgraph-checkpoint) (3.0.0)
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.3.45->langsmith) (1.0.0)
Requirement already satisfied: requests>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.3.45->langsmith) (2.32.0)
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.3.45->langsmith) (0.23.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.0.0->langsmith) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.0.0->langsmith) (2.3.0)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (from anyio->httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2.2->langgraph-sdk) (1.3.1)
```

```
from langgraph.graph import StateGraph, END
import requests

# Replace with your Lambda API Gateway URL
lambda_url = "https://2ooay23h0c.execute-api.us-east-1.amazonaws.com/default/LangGraphStaticData"

# Send GET request
response = requests.get(lambda_url)

# Check status
if response.status_code == 200:
    data = response.json()
    print("✅ Data received from Lambda:")
    print(data)
else:
    print("❌ Error:", response.status_code, response.text)
```

```
✅ Data received from Lambda:
{'products': [{'id': 1, 'name': 'Laptop', 'price': 800}, {'id': 2, 'name': 'Headphones', 'price': 150}, {'id': 3, 'name': 'Smartwatch', 'price': 250}]}
```

```
from typing import TypedDict, Any
from langgraph.graph import StateGraph, END
import requests

# Define the state schema
class GraphState(TypedDict):
    """
    Represents the state of our graph.

    Attributes:
        lambda_data: Data fetched from the Lambda function.
    """
    lambda_data: Any

# Define the node function to fetch data from Lambda
def fetch_lambda_data(state: GraphState):
    """
    Fetches data from the Lambda API Gateway.

    Args:
        state (GraphState): The current graph state.

    Returns:
        dict: The updated state with fetched data.
    """
    lambda_url = "https://2ooay23h0c.execute-api.us-east-1.amazonaws.com/default/LangGraphStaticData"
    response = requests.get(lambda_url)
```

```
response = requests.get(lambda_url)

if response.status_code == 200:
    data = response.json()
    print("✅ Data received from Lambda:")
    print(data)
    return {"lambda_data": data}
else:
    print("❌ Error:", response.status_code, response.text)
    return {"lambda_data": None}

# Create a StateGraph instance with the defined schema
graph = StateGraph(GraphState)

# Add the fetch_lambda_data function as a node
graph.add_node("fetch_lambda_data", fetch_lambda_data)

# Set the entry point to the fetch_lambda_data node
graph.set_entry_point("fetch_lambda_data")

# Add an edge from fetch_lambda_data to END
graph.add_edge("fetch_lambda_data", END)

# Compile the graph
compiled_graph = graph.compile()

# Invoke the graph
final_state = compiled_graph.invoke({"lambda_data": {}})

# Print the fetched Lambda data
print("Final state:", final_state["lambda_data"])
```

```
✅ Data received from Lambda:
{'products': [{'id': 1, 'name': 'Laptop', 'price': 800}, {'id': 2, 'name': 'Headphones', 'price': 150}, {'id': 3, 'name': 'Smartwatch', 'price': 200}]}
Final state: {'products': [{'id': 1, 'name': 'Laptop', 'price': 800}, {'id': 2, 'name': 'Headphones', 'price': 150}, {'id': 3, 'name': 'Smartwatch', 'price': 200}]}
```